

JobMatch: An Agentic Multi-Role Platform for Explainable Job–Candidate Matching

First Author^{1*}

^{1*}Department of Computer Science, University Placeholder, City, Country.

Corresponding author(s). E-mail(s): author@example.edu;

Abstract

Job–candidate matching platforms must rank applicants against structured requirements while preserving human oversight of hiring decisions. We present JobMatch, an agentic multi-role platform in which Candidate, Employer, and Matchmaking agents coordinate profile ingestion, embedding, and read-only hybrid scoring across separate portals. The Matchmaking agent fuses semantic similarity, skill overlap, experience and compensation compatibility, and location signals into a fixed composite score, then attaches structured explanations to each ranked pair. Agents communicate through a shared event bus; the Matchmaking layer remains read-only with respect to profile stores. Role-aware React interfaces gate workflows on authenticated ownership; users review parsed profiles, inspect explanations, and explicitly save or apply to matches rather than receiving automated decisions.

We evaluate offline on a labeled corpus of 30 resumes, 15 job postings, and 47 graded relevance pairs using macro-averaged precision, recall, and nDCG@5. Compared methods include TF–IDF and BM25 lexical baselines, dense embedding retrieval, reciprocal rank fusion, learned fusion, and cross-encoder reranking. Exhaustive resume-to-jobs ranking at $K=5$ shows multimodal soft-embed and learned-fusion configurations reach nDCG@5 of 0.969 and 0.968 in committed benchmark artifacts, while a five-signal composite ablation reaches 0.942 under a documented weight vector. Lexical baselines remain competitive on this small corpus. Cross-encoder reranking reduces nDCG@5 by 0.108 at substantially higher latency and remains disabled in the default portal configuration. Hard-negative mining yields 150 candidate pairs with zero label conflicts. A synthetic counterfactual fairness audit flags six of ten fabricated profile pairs for rank or score instability; this is an engineering sanity check, not validation of demographic fairness in live hiring.

Benchmark drivers, regression tests, and timestamped JSON artifacts reproduce reported metrics. We document architectural realization, scoring formalism, and

Fig. 1: End-to-end agentic hiring platform architecture. Role portals invoke a thin API gateway; domain agents own profiles and embeddings; the Matchmaking agent performs read-only hybrid scoring and explanation. Dashed edges denote offline or optional paths.

evaluation methodology so that offline results can be traced to deployed code paths. The contribution is an integrated research prototype with reproducible benchmarks rather than a validated production hiring system; we do not claim effectiveness beyond the demo corpus or demographic fairness in operational use.

Keywords: multi-agent systems, job matching, explainable AI, information retrieval, hybrid ranking

1 Introduction

Recruiting workflows combine document understanding, constraint checking, and ranked retrieval [1], yet many deployed systems expose only opaque scores to candidates and employers. JobMatch addresses this gap with a *role-aware agentic platform* [2]: dedicated agents own candidate and job profiles, a read-only Matchmaking agent scores and explains matches, and separate web portals keep hiring decisions with human users.

This paper makes three contributions aligned with the implemented system. First, we describe an architectural pattern that separates profile mutation from matchmaking and uses an in-process event bus to invalidate match caches when profiles change. Second, we specify a composite matching model that fuses semantic, skill, title, experience, compensation, and remote-preference signals with rule-based explainability attached to every `MatchResult`. Third, we document an offline evaluation protocol—including negative mining, regression gates, and synthetic fairness audits—with metrics reported only from committed benchmark artifacts.

The remainder of the paper is organized as follows. Section 2 presents platform architecture and portal realization; Section 3 formalizes representation and ranking; Section 4 defines the evaluation framework; Section 5 reports measured results; Section 6 discusses implications and limitations; Section 7 concludes.

2 System Architecture

JobMatch is organized as a role-aware agentic hiring platform with three collaborating agents, a shared event bus, and separate candidate, employer, and admin experiences. Figure 1 summarizes the end-to-end architecture; Figures 2 and 3 show representative candidate and employer workflows.

2.1 Platform overview

Presentation-layer portals (candidate, employer, admin) call a FastAPI gateway with session authentication. Candidate and Employer agents own profile ingestion, embedding, and vector persistence. The Matchmaking agent reads snapshots only, invokes the matching core, and returns ranked, explainable results. SQLite stores auth links, feedback, and activity; an offline benchmark module regenerates paper tables.

2.2 Architectural realization

The deployed application separates concerns by *role portal*, not by microservice boundary: a single React SPA mounts three route trees, each wrapped in a role guard and a shared `PortalShell` layout. This subsection documents how that separation appears to users and which gateway endpoints each workflow invokes.

Role-aware presentation layer.. Three layouts define navigation and visual context (`data-portal` attribute on the document root):

- **Candidate** (`CandidateLayout`): *Resume* (onboarding), *Profile*, and *Jobs* (matches plus saved/applications under the same nav highlight).
- **Employer** (`EmployerLayout`): *My jobs*, *Candidates* (reverse match), and *Applicants*.
- **Admin** (`AdminLayout`): a single *Console* route for operator tooling.

React Router nests each tree under `ProtectedRoute` with an explicit `role` prop; mismatched roles receive HTTP-style 403 pages rather than silent data leakage. After login, `AuthContext` redirects to a role home path (`/candidate/matches`, `/employer/jobs`, or `/admin/console`).

Authentication and ownership.. Registration selects one of three roles (candidate, employer, admin) stored in SQLite (`auth/store.py`). Login establishes a server-side session cookie; the frontend axios client sends `withCredentials: true` on all API calls. Gateway handlers enforce role with `require_role(...)` dependencies. Candidate endpoints such as `GET /candidates/me` resolve the authenticated user to a `candidate_id` via `candidate_ownership`; employer job mutations require a row in `job_ownership`. Embeddings and vector indices are not exposed in public profile JSON (stripped in `_public_profile`).

Resume and job-description ingestion.. Parsing is *assistive*: extracted fields populate forms; users confirm before an agent registers a snapshot.

Candidate path. Onboarding uploads PDF or text via `POST /candidates/upload-resume`. The gateway extracts plain text (`extract_text_from_upload`), then calls `parse_resume_document` with an LLM-backed parser when configured. The UI shows `ExtractedSectionsPanel`, parse status, and `ProfileQualityPanel` hints. Saving calls `PUT /candidates/me`, which triggers the Candidate agent to normalize skills, embed document text, upsert the vector store, and emit a `PROFILE_UPDATED` bus event.

Employer path. Job posting supports file upload (`POST /jobs/upload-description`) or pasted text (`POST /jobs/parse-description`).

Extracted title, skills, experience, budget, and remote policy merge into `JobPostingForm`; `JobQualityPanel` surfaces completeness warnings. Posting invokes `POST /jobs` or updating an owned job via `PUT /jobs/mine/{job.id}`, registering a job snapshot with the Employer agent.

Match discovery and explainability consumption.. Candidate and employer match pages call `POST /match/candidate-to-jobs` or `POST /match/job-to-candidates` with shared defaults: `strategy=composite`, `topK=10`, `skillsMode=jaccard`, calibration and feedback boost disabled (`frontend/src/api/client.js`). The candidate matches view gates search on profile readiness (complete fields, non-stale snapshot) and refreshes when `PROFILE_UPDATED` fires cross-page.

Each `MatchResult` row may include a structured `explanation` object from `core/match_explanation.py`. The frontend resolves explanations in `matchExplainability.js`: prefer server payload, otherwise synthesize fit labels from legacy score fields for backward compatibility. `MatchExplainability` renders two densities:

- **Compact** (result cards): matched/missing skill chips, four fit rows (experience, compensation, remote, semantic), optional mini score breakdown.
- **Full** (`MatchDetailsDrawer`): expanded breakdown bars with component weights matching the composite formula (28/27/10/15/10/10).

Client-side filters (e.g., remote-friendly) read `explanation.remote.score` rather than re-implementing constraint logic. Users act on matches explicitly (save, apply, contact, reject) via activity and feedback endpoints; ranking does not auto-apply on their behalf.

Admin console.. `AdminConsole` aggregates operator views not exposed to hiring roles: live agent status (`GET /agents/status`), recent bus events (`GET /agents/events/recent`), runtime config and demo reset (`GET/POST /system/*`), and proxy-group fairness summaries (`GET /system/fairness`). `MatchControls` allows manual match and daily-batch invocations for inspection; results render in the same `ResultsPanel` component used for debugging ranked lists. This console validates agent wiring; it is not the primary employer or candidate experience.

Table 1 summarizes the main end-user flows and their backend touchpoints.

2.3 Candidate workflow

2.4 Employer workflow

2.5 Matching and evaluation pipelines

Figure 4 details the matching pipeline from retrieval through composite scoring, optional fusion, and explanation generation. Figure 5 shows the offline evaluation pipeline that aggregates IR metrics, fairness audits, and reproducible tables.

Table 1: Primary user-facing workflows by role ($K=10$ composite matching unless noted).

Role	Portal route	User goal	API / agent path
Candidate	<code>/candidate/onboarding</code>	Upload resume, review extraction, create profile	POST <code>/candidates/upload-resume</code> → PUT <code>/candidates/me</code> → Candidate agent registers snapshot + embedding
Candidate	<code>/candidate/profile</code>	Edit skills, experience, pay, remote prefs	GET/PUT <code>/candidates/me</code> ; quality via POST <code>/candidates/quality-check</code>
Candidate	<code>/candidate/matches</code>	View ranked jobs, filter, explain scores	POST <code>/match/candidate-to-jobs</code> (composite); profile readiness gate before search
Candidate	<code>/candidate/saved</code>	Saved jobs and applications	GET/PUT <code>/candidates/me/saved-jobs</code> ; POST <code>/candidates/me/applications</code>
Employer	<code>/employer/jobs</code>	Import JD, post/edit/close roles	POST <code>/jobs/upload-description</code> or <code>/parse-description</code> → POST <code>/jobs</code> or PUT <code>/jobs/mine/{id}</code> → Employer agent
Employer	<code>/employer/matches</code>	Rank candidates for a owned job	POST <code>/match/job-to-candidates</code> (composite); job ownership required
Employer	<code>/employer/applications</code>	Review applicants per posting	GET <code>/jobs/mine/applications</code> (SQLite activity store)
Admin	<code>/admin/console</code>	Monitor agents, run matches, inspect fairness	GET <code>/agents/status</code> , <code>/agents/events/recent</code> , <code>/system/config</code> ; optional <code>/match/*</code> and <code>/system/fairness</code>

All authenticated routes use cookie sessions (`withCredentials`) and FastAPI `require_role` guards. Match endpoints invoke the Matchmaking agent read-only over snapshots.

3 Representation, Retrieval, and Ranking

This section specifies how JobMatch represents candidates and jobs, retrieves candidates from a shared corpus, and produces ranked, explainable match scores. The Matchmaking agent is read-only with respect to profile stores: it consumes immutable *snapshots* produced by the Candidate and Employer agents after parsing and embedding. Figure 4 summarizes the scoring pipeline; portal defaults use the *composite* strategy unless the client overrides it.

3.1 Entity representation

Each side of the market is modeled at two levels: a mutable *profile* (editable portal record) and an immutable *snapshot* (versioned input to matching).

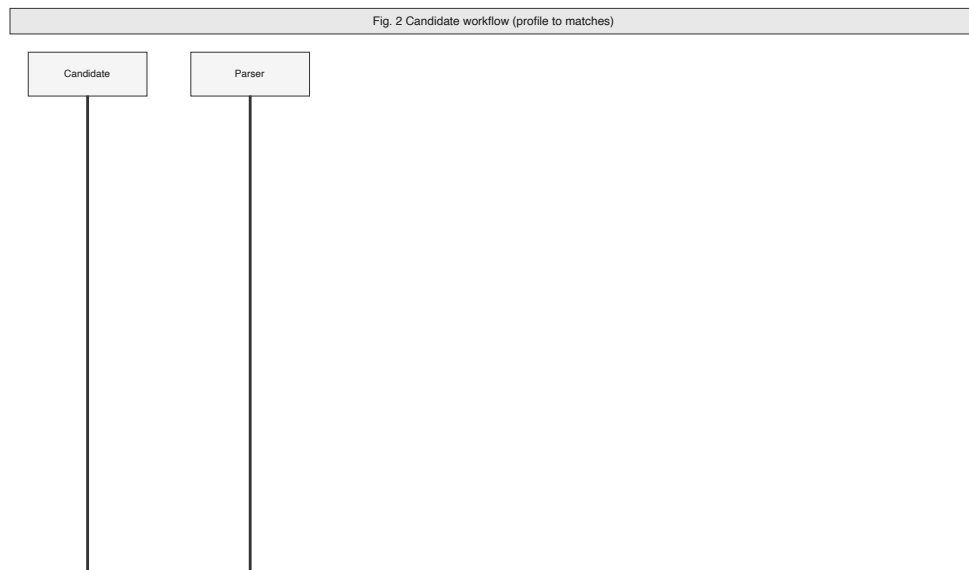


Fig. 2: Candidate workflow from resume update to ranked job matches. Parsing is assistive; the user reviews extracted fields before the profile snapshot is registered.

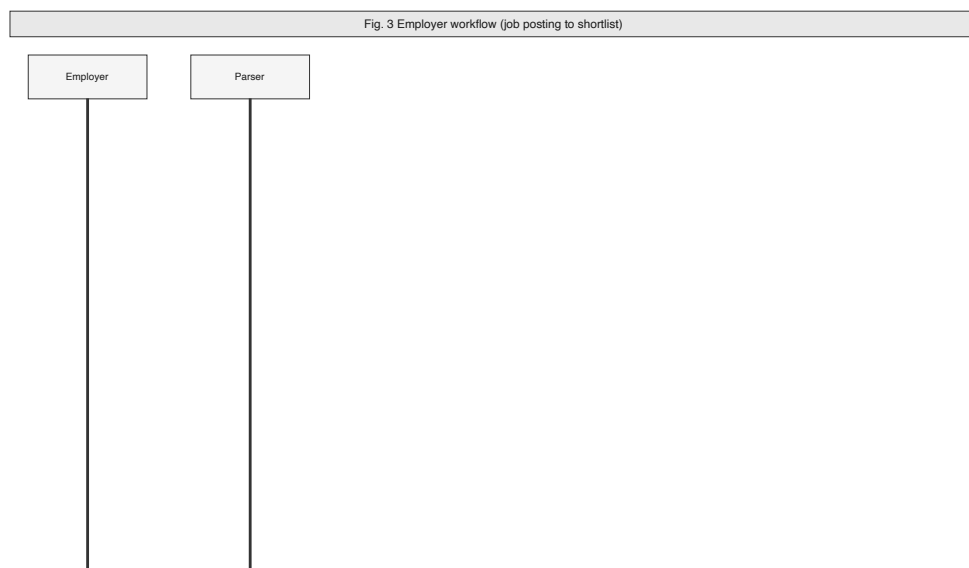


Fig. 3: Employer workflow from job description upload to candidate shortlist. Fairness checks run in the offline audit pipeline; the portal surfaces explanations for human review.

Fig. 4 Matching pipeline (hybrid retrieval and scoring)

Fig. 4: Matching pipeline combining lexical and embedding retrieval with constraint-aware composite scoring, optional fusion and calibration, and structured explanation generation.

Fig. 5 Offline evaluation pipeline

Fig. 5: Offline evaluation pipeline producing IR metrics, fairness audit reports, and reproducible LaTeX tables from labeled corpus pairs.

Candidate side.. A **candidate profile** P_c stores identity fields, structured skills, experience, compensation expectation, remote preference, summary text, and a canonical `document_text` derived from the resume. When the profile is registered or updated, the Candidate agent builds a **candidate snapshot** σ_c containing:

- identifier, display name, normalized skill list;
- scalar constraints: `experience_years`, `preferred_salary`, `remote_preference`;
- `summary`, monotonic `version`, and `document_text_hash`;
- a fixed-dimensional embedding vector $\mathbf{e}_c \in \mathbb{R}^d$ ($d=384$ via `all-MiniLM-L6-v2`).

Snapshots are the only candidate objects passed to scoring and retrieval.

Employer side.. A **job posting** P_j (job profile) stores title, description, required and preferred skills, experience requirement, budget bounds, remote policy, and metadata. The Employer agent emits a **job snapshot** σ_j with the parallel fields needed for matching: `required_skills`, `preferred_skills`, `required_experience`, `budget/budget_min/budget_max`, `remote_policy`, `description`, version hash, and embedding $\mathbf{e}_j$.

Activity and feedback events.. Portal actions are persisted separately from snapshots:

- **Match feedback** (`save`, `dismiss`, `apply`) keyed by `(candidate_id, job_id)` in SQLite;
- **User feedback** (broader actions including `not_interested`, `contact`, `reject`) keyed by user, target, and optional context;
- **Candidate activity** (saved jobs, applications, employer shortlist entries) for demo and audit trails.

Feedback records are event logs; they do not mutate embeddings or snapshots unless a new profile version is submitted.

3.2 Text representation

Raw resume and job-description text are normalized before embedding. Skills pass through a catalog that canonicalizes aliases (e.g., `k8s`→`kubernetes`) so lexical and set-based scores operate on comparable tokens.

For a resume r and job description d we maintain:

- **Resume text** T_r : concatenation of summary, experience narrative, and skill phrases;
- **JD text** T_d : title plus description body;
- **Structured skills** $\mathcal{S}_c$ and $\mathcal{S}_j^{\text{req}}$, $\mathcal{S}_j^{\text{pref}}$ extracted or edited in the portal;
- **Constraint fields** used both as hard feasibility checks and soft score components: experience years vs. required experience, remote preference vs. remote policy, and expected salary vs. posted budget band.

Document embeddings are $\mathbf{e}_c = f_\theta(T_r)$ and $\mathbf{e}_j = f_\theta(T_d)$ with a shared bi-encoder f_θ .

3.3 Retrieval

Matching supports two retrieval regimes.

Dense embedding retrieval (portal default for large pools).. Given query embedding $\mathbf{q}$ (candidate resume embedding for job search, or job embedding for candidate search), the vector store returns the top- k neighbors under approximate nearest-neighbor (ANN) search:

$$\mathcal{N}_k(\mathbf{q}) = \arg \max_{j \in \mathcal{J}, |\cdot|=k} \text{sim}(\mathbf{q}, \mathbf{e}_j), \quad (1)$$

where sim is cosine similarity (default) or a derived Euclidean score (see §3.4.1). Implementations plug in behind a `VectorStorePort` interface (`upsert`, `search`, `delete`); production supports Chroma and Qdrant backends selected at runtime.

Lexical retrieval (offline benchmarks).. Research baselines include BM25 and TF-IDF cosine over tokenized document text, plus exact skill-overlap ranking. These baselines isolate lexical signal from embedding retrieval and are not enabled in the default portal API.

Exhaustive reranking.. When the candidate pool is small, the Matchmaking agent may score all job snapshots exhaustively after optional ANN shortlisting. Pool size and retrieval mode (`exhaustive` vs. `ann`) are request parameters.

3.4 Scoring

All component scores are normalized to $[0, 1]$. The portal default **composite** strategy combines six signals with fixed weights w_s (summing to 1):

$$s_{\text{final}} = w_{\text{sem}} s_{\text{sem}} + w_{\text{skl}} s_{\text{skl}} + w_{\text{tit}} s_{\text{tit}} + w_{\text{exp}} s_{\text{exp}} + w_{\text{comp}} s_{\text{comp}} + w_{\text{rem}} s_{\text{rem}}, \quad (2)$$

with $(w_{\text{sem}}, w_{\text{skl}}, w_{\text{tit}}, w_{\text{exp}}, w_{\text{comp}}, w_{\text{rem}}) = (0.28, 0.27, 0.10, 0.15, 0.10, 0.10)$. The result is clipped to $[0, 1]$.

3.4.1 Semantic similarity

Let $\mathbf{e}_c, \mathbf{e}_j \in \mathbb{R}^d$ be snapshot embeddings. Cosine semantic score:

$$s_{\text{sem}}^{\text{cos}} = \frac{\mathbf{e}_c \cdot \mathbf{e}_j}{\|\mathbf{e}_c\| \|\mathbf{e}_j\|}. \quad (3)$$

Euclidean-derived similarity (alternative metric):

$$s_{\text{sem}}^{\text{euc}} = \frac{1}{1 + \|\mathbf{e}_c - \mathbf{e}_j\|_2}. \quad (4)$$

3.4.2 Skill overlap

Let $A = \text{canon}(\mathcal{S}_c)$ and $B = \text{canon}(\mathcal{S}_j^{\text{req}})$ be canonical skill sets. **Jaccard** mode (default in portals):

$$s_{\text{skl}}^{\text{jac}} = \begin{cases} 0 & \text{if } A = \emptyset \text{ or } B = \emptyset, \\ \frac{|A \cap B|}{|A \cup B|} & \text{otherwise.} \end{cases} \quad (5)$$

Soft embedding mode averages, for each required skill $b \in B$, the maximum cosine similarity to any resume skill embedding:

$$s_{\text{skl}}^{\text{soft}} = \frac{1}{|B|} \sum_{b \in B} \max_{a \in A} \cos(f_{\theta}(a), f_{\theta}(b)). \quad (6)$$

Hierarchical blending of must-have vs. preferred skills is supported offline with weight $\lambda=0.7$ on required coverage.

3.4.3 Constraint compatibility

Experience, compensation, remote, and title components are piecewise mappings from structured fields to $[0, 1]$. Experience score s_{exp} decreases as the gap (`required_experience - experience_years`) grows beyond zero. Compensation score s_{comp} rewards salary expectations inside the posted budget band and decays with overshoot ratio. Remote score s_{rem} penalizes remote-seeking candidates against on-site-only roles. Title score s_{tit} blends token coverage and Jaccard overlap between job-title tokens and candidate summary/skills.

Optional hard constraints.. When `apply_constraints=true`, a multiplicative feasibility factor $\phi \in (0, 1]$ adjusts the composite score for experience gaps, remote mismatch, salary overshoot, and insufficient must-have skill coverage:

$$s_{\text{adj}} = \text{clip}_{[0,1]}(\phi \cdot s_{\text{final}}). \quad (7)$$

Constraint notes are surfaced to the explainer; they do not automate hire/reject decisions.

3.4.4 Multimodal weighted fusion (two-signal baseline)

The **multimodal** strategy blends semantic and skill scores with weight α (default $\alpha=0.7$):

$$s_{\text{mm}} = \alpha s_{\text{sem}} + (1 - \alpha) s_{\text{skl}}. \quad (8)$$

3.5 Fusion and strategy routing

JobMatch supports three fusion layers beyond Eq. (2).

Reciprocal rank fusion (RRF).. Given ranked lists $\mathcal{L}_1, \dots, \mathcal{L}_m$ from alternate strategies (e.g., semantic cosine, multimodal Jaccard), RRF assigns:

$$\text{RRF}(x) = \sum_{i=1}^m w_i \sum_{r=1}^{|\mathcal{L}_i|} \mathbb{I}[\mathcal{L}_i[r] = x] \cdot \frac{1}{k + r}, \quad (9)$$

with rank smoothing constant $k=60$ and optional per-list weights w_i . RRF is used in ensemble endpoints and offline progression studies; the default candidate portal uses composite scoring rather than RRF.

Learned and hierarchical fusion.. A learned fusion model and hierarchical must/preferred skill blend are available for research runs (`fusion_mode` $\in$ `{learned, hierarchical}`). *TODO*: Report offline nDCG@5 for learned fusion with current composite weights.

Strategy router.. When `auto_strategy=true`, profile features route to a strategy triple (strategy, skills_mode, α): skill-rich profiles ($|\mathcal{S}_c| \geq 8$) prefer multimodal soft-embed; moderate skill lists use multimodal Jaccard; sparse profiles fall back to semantic cosine. Routing reasons are attached to the score breakdown for audit.

3.6 Calibration

Raw match scores are not guaranteed to be calibrated probabilities of relevance. An optional **Platt calibrator** maps score s to a calibrated value $\hat{p}$:

$$\hat{p}(s) = \sigma(as + b) = \frac{1}{1 + e^{-(as+b)}}, \quad (10)$$

with parameters (a, b) fit offline on labeled eval pairs via gradient descent (`benchmarks/train_ml_models.py`). Calibration is *disabled* in default portal requests (`use_calibration=false`) and enabled in selected fairness and research pipelines.

Confidence bands.. The explainer maps calibrated or raw s_{final} to discrete fit labels for UI presentation: Strong (≥ 0.85), Good (≥ 0.65), Moderate (≥ 0.45), Weak (< 0.45). These bands aid human interpretation; they are not automated decision thresholds. *TODO*: Report calibration error (ECE or Brier) on held-out eval pairs.

3.7 Explainability

Every `MatchResult` carries a structured `MatchExplanation` payload:

- **Matched skills** and **missing skills** from canonical set overlap;
- **Fit signals** for semantic, experience, compensation, and remote dimensions, each with score, label, and natural-language *reason*;
- **Score breakdown**: per-component weight, raw score, and weighted contribution (for composite strategy).

A rules-based explainer generates bullet rationales; an optional grounded template explainer adds calibrated-relevance phrasing when calibration is active. Explainability is assistive: recruiters and candidates review explanations before save, apply, or contact actions.

3.8 Fairness

JobMatch does **not** ingest or infer protected demographic attributes for ranking. Fairness analysis is limited to *bias-aware offline evaluation*:

- **Proxy-group audit**: experience tier (junior/mid/senior) and remote-seeking vs. onsite-flexible groups; reports selection-rate parity and disparate-impact ratio on ranked lists;
- **Synthetic counterfactual audit**: controlled profile perturbations in `fairness_audit_profiles.json` to flag rank instability under non-demographic edits and fabricated demographic-like pairs (research only).

Neither audit automates hiring decisions or demographic classification in production. *TODO*: Insert latest disparate-impact ratios from `fairness_audit_report.json` after regeneration with current composite weights.

3.9 Feedback signals

Interaction feedback is stored when users save, dismiss, or apply to a match. An optional **feedback boost** adjusts the final score by

$$s' = \text{clip}_{[0,1]}(s + n_{\text{save}} \cdot 0.04 + n_{\text{apply}} \cdot 0.06 - n_{\text{dismiss}} \cdot 0.06), \quad (11)$$

where n_{save} , n_{apply} , and n_{dismiss} are historical action counts for the pair. Feedback boost is *off by default* (`use_feedback_boost=false`) so portal rankings remain reproducible from snapshots alone. User-feedback rows additionally capture employer actions (`contact`, `reject`) for analytics but do not alter ranking unless feedback boost is explicitly enabled.

3.10 Evaluation linkage

Offline metrics use graded relevance labels on 30 candidate queries $\times$ 15 jobs (47 labeled pairs, scale 0–2). Report $\text{precision}@K$, $\text{recall}@K$, and $\text{nDCG}@K$ at $K=5$ unless noted.

Verified benchmark snapshots (2026-05-27 run).. These numbers come from docs/research/evaluation/ and use the research driver configuration; they may differ from live portal defaults when weights or flags differ:

- Multimodal weighted ($\alpha=0.7$, Jaccard skills): nDCG@5 = 0.924, P@5 = 0.287, R@5 = 0.933;
- RRF ensemble (four strategy lists, $k=60$): nDCG@5 = 0.913, P@5 = 0.293, R@5 = 0.950;
- Full composite ablation (five components, *legacy* weights 0.40/0.30/0.15/0.10/0.05, no title term): nDCG@5 = 0.942, P@5 = 0.307, R@5 = 0.983.

TODO: Re-run ablation with production composite weights (0.28/0.27/0.10/0.15/0.10/0.10 including title) and update Table reftab:ablation. *TODO:* Report feedback-boost and calibration-on vs. calibration-off deltas on the eval corpus.

4 Evaluation Framework

This section defines the offline protocol used to validate JobMatch ranking quality, fusion variants, and fairness sanity checks. All metrics are computed on a fixed labeled corpus; production portal traffic is not used for the numbers reported here. Figure 5 summarizes the evaluation pipeline from labeled pairs through metric aggregation and regression gates.

4.1 Dataset and relevance labels

We evaluate *resume-to-jobs* retrieval: each of 30 candidate resumes is a query, and the corpus contains 15 job postings. Graded relevance judgments are stored in `data/eval_pairs.json` (version 1.0) with scale $r \in \{0, 1, 2\}$ (none, partial, strong). The loader (`benchmarks/eval_data.py`) merges duplicate (query, doc) entries by taking the maximum grade and discards negative placeholders.

For each query q , let $\text{rel}_q(j)$ denote the graded relevance of job j . The *relevant set* for binary metrics is $\mathcal{R}_q = \{j : \text{rel}_q(j) > 0\}$. The corpus contains 47 graded pairs covering all 30 queries (1–2 relevant jobs per query: 21 strong and 26 partial labels). Table 2 summarizes corpus statistics.

Snapshots for evaluation are built with the same code path as the agents: canonical document text, SHA-256 hash, and `all-MiniLM-L6-v2` embeddings (§3.2).

4.2 Ranking metrics

Given a ranked list $\pi_q = (j_1, j_2, \dots)$ for query q , let $\pi_q[:K]$ denote the top- K prefix. All benchmark drivers default to $K=5$ unless noted. Metrics are implemented in `benchmarks/metrics.py` and macro-averaged over queries present in both the label file and the ranking output.

Table 2: Evaluation corpus statistics.

Statistic	Value
Task	Resume $\rightarrow$ jobs (candidate-initiated retrieval)
Candidates (queries)	30
Job postings (corpus)	15
Graded query-document pairs	47
Queries with ≥ 1 relevant job	30
Relevance scale	0 (none), 1 (partial), 2 (strong)
Grade-2 (strong) labels	21
Grade-1 (partial) labels	26
Labels per query (min / max)	1 / 2
Embedding model	<code>all-MiniLM-L6-v2</code> ($d=384$)
Label source	<code>data/eval_pairs.json</code> v1.0

Starter labeled set for offline benchmarks; not derived from live portal traffic.

Precision and recall..

$$P@K(q) = \frac{1}{K} \sum_{j \in \pi_q[:K]} \mathbb{1}[j \in \mathcal{R}_q], \quad (12)$$

$$R@K(q) = \frac{1}{|\mathcal{R}_q|} \sum_{j \in \pi_q[:K]} \mathbb{1}[j \in \mathcal{R}_q]. \quad (13)$$

Discounted cumulative gain (nDCG@K).. Graded nDCG uses gain $2^r - 1$ at each rank:

$$DCG@K(q) = \sum_{i=1}^K \frac{2^{\text{rel}_q(j_i)} - 1}{\log_2(i+1)}, \quad \text{nDCG@K}(q) = \frac{DCG@K(q)}{DCG@K(q^*)}, \quad (14)$$

where π_{q^*} is the ideal ranking by descending rel_q .

Auxiliary metrics.. Mean reciprocal rank (MRR) and mean average precision (MAP) treat relevance as binary ($r > 0$). Bootstrap significance tests (`benchmarks/significance.py`, 5000 resamples, seed 42) compare method pairs on per-query nDCG@5 and MRR with one-sided p -values.

Latency.. When reported, latency is mean wall-clock milliseconds per query on a single machine (`comparison_summary.json`, `cross_encoder_report.json`). *TODO:* Commit `phase11_summary.csv` from `benchmarks/phase11.py` for ANN shortlist + rerank latency (pool=10, Chroma/Qdrant grid).

Fairness metrics.. Two offline audits complement ranking IR metrics:

- **Proxy-group parity** (`core/fairness.py`, `fairness_eval.py`): experience tier and remote-preference groups; reports top- K selection rate vs. corpus median and disparate-impact ratio $\min_g \text{rate}_g / \max_g \text{rate}_g$.

Table 3: Retrieval method comparison (macro-averaged, $K=5$).

Method	Family	P@5	R@5	MRR	nDCG@5	MAP
Multimodal weighted	Embedding	0.287	0.933	0.961	0.924	0.867
RRF ensemble	Embedding	0.293	0.950	0.944	0.913	0.845
TF-IDF cosine	Lexical	0.293	0.950	0.918	0.898	0.842
BM25	Lexical	0.307	0.983	0.912	0.894	0.838
Semantic cosine	Embedding	0.267	0.867	0.931	0.878	0.810
Semantic euclidean	Embedding	0.267	0.867	0.931	0.878	0.810
Soft skill embed	Embedding	0.280	0.900	0.911	0.869	0.829
Exact skill overlap	Lexical	0.233	0.733	0.816	0.748	0.681
Skills Jaccard	Embedding	0.233	0.733	0.816	0.748	0.681

Source: research pipeline run 2026-05-27T150000Z/comparison/comparison_summary.json.
Exhaustive ranking over all 15 jobs per query. Latency in Table 9.

- **Synthetic counterfactual audit** (`benchmarks/fairness_audit.py`): 10 fabricated profile pairs; flags rank instability and score drift under controlled field edits (Table 11).

Neither audit uses protected attributes from real users. *TODO*: Persist `fairness_eval.py` proxy-group ratios to `benchmark_outputs/`.

4.3 Compared methods and drivers

Benchmarks are organized into drivers that share `eval_rankings()` but differ in how π_q is produced:

- **Comparison study** — lexical (BM25, TF-IDF, exact overlap) vs. embedding (semantic, Jaccard, soft embed, multimodal, RRF); exhaustive ranking over 15 jobs.
- **Composite ablation** — single-signal, partial, and full composite variants with documented weight vectors.
- **Fusion ablation** (`table11_fusion.py`) — fixed multimodal, hierarchical skills, learned logistic-regression fusion, constraint-adjusted multimodal.
- **Paper progression** (`paper_progression.py`) — ordered lexical $\rightarrow$ semantic $\rightarrow$ multimodal $\rightarrow$ RRF $\rightarrow$ cross-encoder path for narrative and regression baselines.
- **ANN latency** (`phase11.py`) — ANN shortlist then rerank; *TODO*: no committed summary artifact at time of writing.

Table 3 reports the main method comparison from the timestamped research run (2026-05-27). Table 5 stores regression baselines used by CI (full results in §5).

4.4 Negative mining

Hard negatives support contrastive analysis and future reranker training (`benchmarks/negative_mining.py`). For each labeled query q , the driver scores all 15 jobs with multimodal weighted ranking ($\alpha=0.7$, default Jaccard skills), sorts

Table 4: Paper progression benchmark (regression base-lines, $K=5$).

Method	P@5	R@5	nDCG@5
Multimodal soft embed $w=0.7$	0.313	1.000	0.969
Soft embed + cross-encoder (pool=10)	0.307	0.983	0.939
RRF ensemble (4 lists)	0.293	0.950	0.935
Multimodal Jaccard $w=0.7$	0.293	0.950	0.933
Semantic cosine (rich templates)	0.300	0.950	0.922
Semantic cosine	0.280	0.900	0.911
TF-IDF (lexical)	0.307	0.983	0.905
BM25 (lexical)	0.307	0.983	0.901

Source: `data/expected/paper_progression.summary.json`. Produced by `benchmarks/paper_progression.py`; gated by `tests/benchmarks/test_eval_regression.py` (± 0.04 nDCG tolerance). Values can differ from Table 3 when template or cache flags differ.

descending, and selects up to five jobs from the top- $P=10$ pool that are *not* in $\mathcal{R}_q$:

$$\mathcal{H}_q = \{j \in \pi_q[:P] : j \notin \mathcal{R}_q\}_{|\cdot| \leq 5}. \quad (15)$$

The report is written to `backend/benchmark_outputs/hard_negatives.json` (30 queries, 5 hard negatives each when the pool permits). Hard negatives are diagnostic artifacts; they are not injected into portal ranking unless a future reranker is trained and enabled.

4.5 Regression tests

Reproducibility is enforced by `tests/benchmarks/test_eval_regression.py`. The test reloads `data/eval_pairs.json`, rebuilds snapshots, reruns a subset of ranking methods, and compares macro-averaged nDCG@5 against `data/expected/paper_progression.summary.json` with tolerance $\tau=0.04$:

$$\left| \widehat{\text{nDCG@5}}_{\text{run}} - \text{nDCG@5}_{\text{expected}} \right| \leq \tau. \quad (16)$$

Methods gated today: TF-IDF, BM25, semantic cosine, multimodal Jaccard ($w=0.7$), and multimodal soft embed ($w=0.7$). Updating expected JSON requires a deliberate benchmark refresh after embedding or scoring changes. Results and interpretation appear in §5–§6.

5 Results

We report macro-averaged ranking metrics at $K=5$ on the labeled corpus described in §4. Numbers come from committed benchmark artifacts; we do not extrapolate to production hiring outcomes.

Table 5: Paper progression benchmark (regression baselines, $K=5$).

Method	P@5	R@5	nDCG@5
Multimodal soft embed $w=0.7$	0.313	1.000	0.969
Soft embed + cross-encoder (pool=10)	0.307	0.983	0.939
RRF ensemble (4 lists)	0.293	0.950	0.935
Multimodal Jaccard $w=0.7$	0.293	0.950	0.933
Semantic cosine (rich templates)	0.300	0.950	0.922
Semantic cosine	0.280	0.900	0.911
TF-IDF (lexical)	0.307	0.983	0.905
BM25 (lexical)	0.307	0.983	0.901

Source: `data/expected/paper_progression_summary.json`. Produced by `benchmarks/paper_progression.py`; gated by `tests/benchmarks/test_eval_regression.py` (± 0.04 nDCG tolerance). Values can differ from Table 3 when template or cache flags differ.

5.1 Progression of retrieval strategies

Table 5 summarizes the paper-progression driver (`paper_progression.py`), which exhaustively ranks all 15 jobs per query. Among the configurations in `data/expected/paper_progression_summary.json`, multimodal soft-embed ($\alpha=0.7$) achieves the highest nDCG@5 (0.969) and recall@5 (1.000). Lexical baselines (TF-IDF, BM25) remain competitive on this corpus (nDCG@5 $\approx$ 0.901–0.905), which suggests that keyword overlap still carries signal when the job pool is tiny and skill phrases repeat across postings.

RRF over four ranked lists reaches nDCG@5 0.935, above semantic cosine alone (0.911) but below the best single multimodal variant. Adding a cross-encoder reranker on a pool of 10 raises nDCG@5 to 0.939 in this driver—still below soft-embed without reranking—so second-stage reranking is not uniformly beneficial here.

5.2 Fusion and constraint variants

Table 6 reports the fusion ablation driver (`table11_fusion.py`). Learned logistic-regression fusion achieves nDCG@5 0.968 (P@5 0.307, R@5 0.983), matching the progression benchmark’s best soft-embed run within 0.001 nDCG. Fixed multimodal ($\alpha=0.7$) and hierarchical skill blending produce identical scores (nDCG@5 0.924), indicating that must-have vs. preferred hierarchy does not change rankings on this label set with default weights.

Applying feasibility constraints *after* multimodal scoring lowers nDCG@5 to 0.908 (P@5 0.300, R@5 0.950). Constraint filtering therefore trades graded ranking quality for plausibility penalties, a pattern employers may accept in exchange for fewer clearly infeasible suggestions.

Table 7 lists fitted calibration and fusion parameters. The Platt mapping uses $a=2.078$ and $b=2.112$; the learned fusion model assigns the largest coefficients to experience, remote, and salary factors, with semantic and taxonomy features also weighted above raw skill overlap.

Table 6: Fusion and constraint variants ($K=5$).

Method	P@5	R@5	nDCG@5
Learned fusion (LR)	0.307	0.983	0.968
Fixed multimodal ($\alpha=0.7$)	0.287	0.933	0.924
Hierarchical skills	0.287	0.933	0.924
Multimodal + constraints	0.300	0.950	0.908
Semantic cosine	0.267	0.867	0.878

Source: `backend/benchmark_outputs/table11.fusion.json`
(`benchmarks/table11.fusion.py`).

Table 7: Fitted calibration and fusion model parameters (offline training).

Component	Value
<i>Platt calibrator</i> (<code>data/models/calibration.json</code>)	
a (scale)	2.078
b (shift)	2.112
<i>Learned fusion LR</i> (<code>data/models/fusion.json</code>)	
Top feature weights (logistic)	exp. 0.860, remote 0.853, salary 0.849 semantic 0.537, taxonomy 0.509, skills 0.204 must-have coverage 0.311, bias 1.529

Models trained on labeled eval pairs; used when `use_calibration` or `fusion_mode=learned` is enabled. *TODO*: Report calibration impact on nDCG@5 and expected calibration error.

5.3 Composite ablation and latency

Table 8 (research pipeline, 2026-05-27) ablates composite components under a five-signal weight vector (40/30/15/10/5, no title term). Full composite reaches nDCG@5 0.942, compared with 0.917 for semantic+skills and 0.878 for semantic-only. Single-signal constraint channels alone perform poorly ($\text{nDCG@5} \leq 0.393$), confirming that they are useful only in combination with text and skill features.

Table 9 summarizes measured latency on the same evaluation harness. Multimodal weighted scoring costs ~ 0.26 ms/query exhaustive over 15 jobs; soft skill embedding costs ~ 15.5 ms/query because each skill pair requires an embedding comparison. On the composite cross-encoder experiment (pool=20), nDCG@5 falls from 0.942 to 0.834 while latency rises from 0.41 ms to 142 ms per query—a large quality cost for reranking on this setup.

5.4 Hard-negative mining

Table 10 characterizes the hard-negative report (`negative_mining.py`). All 30 queries yield five negatives each (150 pairs total). None of the mined negatives appear in the graded relevant set for their query, so the miner respects the available labels.

Table 8: Ablation over composite matching components ($K=5$).

Variant	Category	P@5	R@5	MRR	nDCG@5	MAP
Full composite	Full	0.307	0.983	0.944	0.942	0.896
Semantic + skills	Partial	0.287	0.933	0.961	0.917	0.867
Semantic + skills + exp.	Partial	0.287	0.933	0.961	0.917	0.866
Semantic only	Single	0.267	0.867	0.931	0.878	0.810
Skills only	Single	0.233	0.733	0.816	0.748	0.681
Compensation only	Single	0.187	0.567	0.457	0.393	0.387
Location only	Single	0.167	0.467	0.388	0.335	0.328
Experience only	Single	0.160	0.467	0.384	0.326	0.314
Component RRF	Ensemble	0.233	0.717	0.570	0.564	0.497

Source: 2026-05-27T150000Z/ablation/ablation.summary.json. Weights for full composite in this run: semantic 40%, skills 30%, experience 15%, compensation 10%, location 5% (no title term). *TODO:* Re-run with portal weights 28/27/10/15/10/10.

Table 9: Quality-latency trade-off (exhaustive evaluation, $K=5$).

Method	nDCG@5	Latency (ms/q)	Notes
Exact skill overlap	0.748	0.03	Lexical
Skills Jaccard	0.748	0.03	Set overlap
TF-IDF cosine	0.898	0.07	Lexical
BM25	0.894	0.11	Lexical
Semantic euclidean	0.878	0.21	Embedding
Semantic cosine	0.878	0.23	Embedding
Multimodal weighted	0.924	0.26	$\alpha=0.7$, Jaccard skills
RRF ensemble	0.913	0.95	Four ranked lists
Soft skill embed	0.869	15.53	Per-skill embedding
Composite (bi-encoder)	0.942	0.41	Ablation full composite
Composite + cross-encoder	0.834	141.7	Pool=20; Δ nDCG=-0.108

Latency from comparison.summary.json and cross.encoder.report.json. *TODO:* Publish phase11.py ANN shortlist latency table (pool=10, Chroma/Qdrant grid).

This does *not* imply the negatives are truly irrelevant: unlabeled jobs are treated as non-relevant by construction. Because the corpus has only 15 jobs and negatives are drawn from the multimodal top-10, hard negatives are “high-scoring unlabeled” pairs—useful for contrastive analysis but not validated against human rejections.

5.5 Fairness audit

Table 11 reports the synthetic counterfactual audit (`fairness.audit.py`). Six of ten fabricated pairs are flagged, mostly when name, hometown, or email fields change. The largest observed score shift is 0.024 (name ethnicity pair); one pair loses top-1 stability when a nationality phrase is appended to the summary.

Table 10: Hard-negative mining summary (multimodal top-10 pool, $K=5$ negatives per query).

Statistic	Value
Queries mined	30
Negatives per query (target / observed)	5 / 5
Total hard-negative pairs	150
Negatives with graded relevance > 0	0
Negatives overlapping declared relevant set	0

Source: `backend/benchmark.outputs/hard_negatives.json`.
Mined with multimodal weighted scoring ($\alpha=0.7$) over 15 jobs; negatives are high-scoring non-labeled jobs, not confirmed irrelevant documents.

Table 11: Synthetic counterfactual fairness audit (10 pairs, composite ranking).

Pair	Field changed	Top-1 stable	Top-5 overlap	Max rank Δ	Max score Δ	Flagged
name_gender_01	name	Yes	5/5	1	0.0096	No
name_gender_02	name	Yes	5/5	0	0.0072	No
name_ethnicity_01	name	Yes	5/5	2	0.0157	Yes
name_ethnicity_02	name	Yes	5/5	3	0.0238	Yes
nationality_phrase_01	summary	No	5/5	1	0.0087	Yes
nationality_phrase_02	summary	Yes	5/5	1	0.0110	No
hometown_01	hometown	Yes	4/5	2	0.0117	Yes
hometown_02	hometown	Yes	5/5	2	0.0143	Yes
pronouns_01	pronouns	Yes	5/5	1	0.0042	No
email_domain_01	email	Yes	4/5	1	0.0082	Yes

Source: `docs/research/evaluation/paper.tables/table4_fairness.csv`. Fabricated profiles only; 6/10 pairs flagged.
TODO: Export proxy-group disparate-impact ratios from `fairness_eval.py`.

These results indicate sensitivity to non-qualification surface text under composite ranking. They do not establish demographic parity in live hiring, and proxy-group disparate-impact ratios from `fairness_eval.py` are not yet committed to `benchmark.outputs/`.

6 Discussion

This section interprets the offline results, states what they do *not* show, and outlines implications for the agentic hiring platform.

6.1 Which ranking strategy performs best?

The answer depends on which benchmark driver and artifact is consulted, and the differences are small relative to corpus size. On the progression baseline (`paper_progression_summary.json`), multimodal soft-embed ($\alpha=0.7$) reaches

nDCG@5 0.969. On the fusion driver (`table11_fusion.json`), learned LR fusion reaches nDCG@5 0.968. On the composite ablation run, full composite reaches nDCG@5 0.942 under a different weight vector than the live portal default.

We therefore do not claim a single universally dominant strategy. What the results support is narrower: combining semantic and skill signals consistently outperforms semantic-only retrieval on this label set, and adding structural constraints (experience, compensation, remote) further helps composite ablations but hurts rankings when applied as a hard post-filter.

The production portal defaults to a six-signal *composite* ranker (28/27/10/15/10/10) rather than soft-embed or learned fusion, prioritizing interpretable fixed weights over the highest offline nDCG observed in research drivers. *TODO*: Publish composite ablation under portal weights for direct deployment alignment.

6.2 Does fusion improve stability?

RRF fusion is intended to stabilize rankings when individual retrievers disagree. In the progression benchmark, RRF (four lists, $k=60$) achieves nDCG@5 0.935, improving over semantic cosine (0.911) but not over the best multimodal single run (0.969). RRF therefore diversifies rankings without guaranteeing a higher graded score on a corpus this small.

Learned fusion fits a logistic model on the same 47 labeled pairs used for evaluation. It reaches nDCG@5 0.968, essentially tied with soft-embed, but inherits the risk of overfitting: training and test queries overlap structurally (same 30 resumes, same 15 jobs). We treat learned fusion as an exploratory upper bound, not a validated production upgrade.

Fixed hierarchical blending duplicates fixed multimodal scores exactly in Table 6, suggesting that preferred-skill tiers are under-specified in the demo corpus.

6.3 Where hard negatives succeed and where they do not

The miner correctly excludes jobs marked relevant in `eval_pairs.json`. Its limitation is epistemic, not mechanical: with 15 jobs and 1–2 labels per query, most jobs are unlabeled. Hard negatives are therefore “top-scoring unlabeled” jobs, which may still be plausible matches a recruiter would shortlist.

We have not measured whether hard negatives confuse a reranker or contrastive trainer; the artifact is descriptive. Extending labels or mining from a larger negative pool would be needed before hard negatives support claims about robustness.

6.4 What calibration changes

Platt calibration applies $\hat{p}(s) = \sigma(2.078s + 2.112)$ to raw match scores when `use_calibration=true`. Because the transform is monotonic, it preserves rank order if applied uniformly; its primary effect is remapping scores onto a $(0, 1)$ interpretability scale for explanations and thresholding.

Portal requests disable calibration by default, so user-visible rankings follow raw composite scores. We do not report expected calibration error or before/after nDCG

for calibrated scores in the current artifact set; calibration should be evaluated separately before using $\hat{p}$ as a decision aid.

6.5 What fairness evaluation shows

The counterfactual audit shows that superficial profile edits can change ranked outputs and composite scores by up to 0.024 on synthetic pairs. Six of ten pairs are flagged under the audit thresholds; four remain unflagged (including both gendered name swaps in the fixture set).

This is a useful engineering sanity check—rankings should not swing on email domains or hometown labels—but it is not a substitute for demographic fairness assessment. No protected attributes are inferred from real applicants; `fairness_eval.py` proxy groups (experience tier, remote preference) are not yet exported to committed JSON. Fairness evaluation is therefore *limited by available labels and synthetic fixtures*, not by absence of code paths.

6.6 Latency–quality trade-offs

Measured latencies favor lightweight scorers on this micro-corpus: BM25 ~ 0.11 ms/query, multimodal ~ 0.26 ms/query exhaustive over 15 jobs. Soft skill embedding is two orders of magnitude slower (~ 15.5 ms/query) with *lower* nDCG@5 (0.869) in the comparison run than multimodal Jaccard (0.924).

Cross-encoder reranking increases latency to ~ 142 ms/query and reduces composite nDCG@5 by 0.108 in the committed cross-encoder report. These measurements argue against enabling cross-encoder rerank in the default portal path, at least for demo-scale corpora. *TODO*: Publish `phase11.py` ANN shortlist latency to separate retrieval from scoring cost at larger pool sizes.

6.7 Agentic workflow interpretation

JobMatch’s agents coordinate ingestion, embedding, and read-only matchmaking, but human users retain save, apply, and hire decisions. The evaluation covers the Matchmaking agent’s scoring core, not end-to-end autonomy: Candidate and Employer agents do not negotiate offers or schedule interviews without user action.

Offline metrics validate ranking components under manual labels; they do not validate that agent event routing improves hiring quality in the field. The appropriate claim is bounded automation with explainable suggestions, not autonomous hiring.

6.8 Limitations

- **Small dataset.** Thirty queries and fifteen jobs yield high recall@5 for many methods (often ≥ 0.95) and narrow nDCG spreads; differences may not replicate at industry scale.
- **Synthetic and manual labels.** Relevance judgments are curated for demo roles, not sampled from applicant tracking systems.
- **No production hiring validation.** We report offline IR metrics only; time-to-hire, applicant satisfaction, and employer acceptance are unmeasured.

- **Fairness scope.** Counterfactual audits use fabricated profiles; proxy-group parity metrics are unreported in committed outputs.
- **Configuration drift.** Ablation composite weights, progression flags (e.g., rich templates), and portal defaults differ across drivers; headline nDCG values are not interchangeable without reading artifact metadata.
- **Learned models fit on eval data.** Fusion and calibration models train on the same labeled pairs used for scoring reports, inflating apparent performance relative to held-out deployment data.
- **Bounded agent autonomy.** Agents synchronize profiles and matches but do not replace recruiter judgment; evaluation does not test multi-session agent negotiation or labor-market dynamics.

In sum, the benchmarks support design choices for a composite, explainable match-maker on a controlled corpus. They do not yet justify strong claims about fairness, efficiency, or hiring outcomes in production.

7 Conclusion

JobMatch demonstrates how a three-agent hiring platform can combine hybrid retrieval, constraint-aware composite scoring, and structured explanations in a deployable monolith with role-separated portals. Offline evaluation on a small labeled corpus shows that multimodal and learned-fusion rankers achieve the highest nDCG@5 values in research drivers, while the production portal favors a fixed composite strategy for interpretability. Cross-encoder reranking and hard constraint post-filters trade measurable ranking quality or latency for other objectives on this dataset.

Future work requires larger labeled corpora, held-out evaluation of learned fusion and calibration, proxy-group fairness exports, and—outside the scope of this artifact—field studies of recruiter and candidate trust in agent-mediated explanations.

Acknowledgments. The authors thank collaborators who reviewed portal workflows and benchmark reproducibility checks.

Declarations

Funding

Not applicable.

Competing interests

The authors declare no competing interests.

Ethics approval

Not applicable. Evaluation uses synthetic and manually curated demo profiles; no human subjects recruitment data were collected for this study.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Data availability

The demo corpus (`data/cvs.json`, `data/jobs.json`, `data/eval_pairs.json`) and benchmark outputs under `backend/benchmark_outputs/` and `docs/research/evaluation/` are included in the project repository. Regenerate metrics with `python backend/scripts/run_research_pipeline.py` and `python -m benchmarks.paper_progression`.

Code availability

Source code for the platform, benchmark drivers, and regression tests is available in the JobMatch repository at the submission tag. Key entry points: `backend/gateway/`, `backend/agents/`, `frontend/src/`, and `tests/benchmarks/test_eval_regression.py`.

Author contributions

First Author: conceptualization, software, evaluation, writing—review and editing. Additional authors: *TODO*: Add CRediT roles before submission.

Prior publication

This manuscript has not been published previously and is not under consideration elsewhere. No portion of the text has appeared in a preprint with colored layout styling.

References

- [1] Manning, C.D., Raghavan, P., Schütze, H.: Introduction to Information Retrieval. Cambridge University Press, ??? (2008)
- [2] Wooldridge, M.: An Introduction to MultiAgent Systems, 2nd edn. Wiley, ??? (2009)